Recursion

AP Computer Science A • Unit 10 • 75-Minute Lesson

Lesson Roadmap

Time	Activity	Focus
0–7 min	Hook + warm-up	The mirror-in-a-mirror problem
7–22 min	Core concept	Definition, base case, recursive case
22–37 min	Worked example 1	Factorial — full call-stack trace
37–50 min	Worked example 2	Fibonacci & string reversal
50–58 min	Recursion vs. iteration	Trade-offs; `StackOverflowError`
58–68 min	AP-style MCQ practice	Tracing & predicting output
68–75 min	Summary + exit card	Big ideas; exam tips

Learning Goals

By the end of this lesson you will be able to:

- identify the two required components of any recursive method: a **base case** and a **recursive case**
- **trace** a recursive method call by hand using a call-stack diagram and determine its return value
- explain why infinite recursion causes a `StackOverflowError`
- compare a recursive solution to an equivalent iterative one and articulate the trade-offs
- correctly answer **AP-style MCQ** questions involving recursion — the most-missed topic on the exam

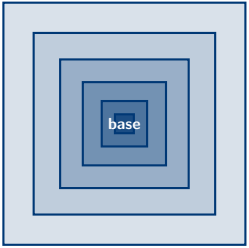
AP CSA CED Connection — Topic 10.1

Learning Objective CON-2.O: Determine the result or output of a recursive method call.
Learning Objective CON-2.P: Explain how a recursive method works.
The AP exam typically includes **4–6 recursion questions** — one of the highest wrong-answer-rate topics.

Hook — Recursion Is Everywhere

Real-world recursive structures:

- A **mirror facing a mirror** — infinite reflections, each containing a smaller copy
- **Russian nesting dolls** (matryoshka) — each doll contains a smaller identical doll until you reach the smallest
- A **dictionary** defines a word using other words, which are defined using other words. . .
- **Fractal geometry** — a coastline looks the same at every scale



Each square contains a smaller square.
Same pattern, smaller scale.

In programming: a method that **calls itself** to solve a problem by breaking it into a **smaller version of the same problem**.

Warm-Up — What Does This Print? (5 min)

```java
public static void countDown(int n) {
    if (n == 0) {
        System.out.println("Blastoff!");
    } else {
        System.out.println(n);
        countDown(n - 1);
    }
}

// Call:
countDown(3);
```

Predict the output before running.
Write it down.

Warm-Up — What Does This Print? (5 min)

```java
public static void countDown(int n) {
    if (n == 0) {
        System.out.println("Blastoff!");
    } else {
        System.out.println(n);
        countDown(n - 1);
    }
}

// Call:
countDown(3);
```

Predict the output before running.
Write it down.

Output

```
3
2
1
Blastoff!
```

Notice: the method calls *itself* (`countDown(n-1)`). Each call uses a *smaller* value of `n`. When `n == 0`, it stops.

What Makes a Method Recursive?

Definition

A method is **recursive** if it contains at least one call to itself as part of its execution.

Every valid recursive method has **exactly two parts**:

1. Base Case

- The condition that **stops** the recursion
- Returns a value **without** making a recursive call
- Must be reachable from any valid input
- A method may have more than one base case

2. Recursive Case

- Makes a call to the **same method**
- Uses a **modified parameter** that moves toward the base case
- Combines the recursive result with additional work
- Must make *progress* — inputs must shrink/simplify

If either component is missing or broken: no base case ⇒ `StackOverflowError`; no progress toward base case ⇒ also `StackOverflowError`.

Anatomy of a Recursive Method

```java
public static int factorial(int n) {

    // BASE CASE
    if (n == 0) {
        return 1;   // stop here
    }

    // RECURSIVE CASE
    return n * factorial(n - 1);
    //       ^    ^^^^^^^^^^^^^^^^^
    //       |    recursive call with
    //       |    smaller argument
    //       combine with result
}
```

Base Case
`n == 0: return 1`

eventually reaches

Recursive Case
`n * factorial(n-1)`

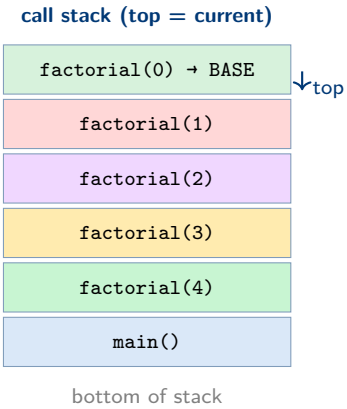
- returns without calling itself
- calls itself with `n-1`
- uses the returned value

The Call Stack — Your Most Important Tool

How Java executes method calls:

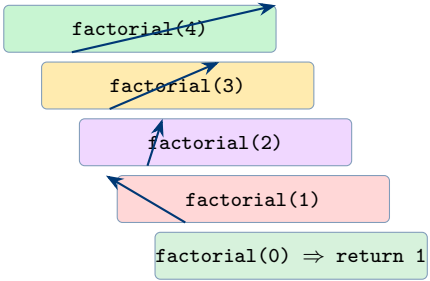
- Each method call creates a new **stack frame**
- The frame holds that call's **local variables** and **parameters**
- Frames are stacked on top of each other (LIFO)
- When a method returns, its frame is **popped**
- Execution resumes in the caller's frame

`StackOverflowError`: the call stack fills up when recursion never reaches a base case. Java's stack has a finite size; too many frames ⇒ crash.



Tracing `factorial(4)` — Winding Down

Phase 1: Recursive calls accumulate (winding down)



Each call suspends and waits for the inner call to return.

Base case returns 1.

Tracing `factorial(4)` — Unwinding Back Up

Phase 2: Return values propagate back (unwinding)

`factorial(0)` **return 1**

↓

`factorial(1)` **return** $1 \times 1 = 1$

↓

`factorial(2)` **return** $2 \times 1 = 2$

↓

`factorial(3)` **return** $3 \times 2 = 6$

↓

`factorial(4)` **return** $4 \times 6 = 24$

return values flow back up the chain

`factorial(4) = 24`

Factorial — Complete Code and Trace Table

```java
public static int factorial(int n) {
    if (n == 0) {
        return 1;                    // base case
    }
    return n * factorial(n - 1);     // recursive
}
```

Call	n	Returns	Computes
factorial(4)	4	24	4×6
factorial(3)	3	6	3×2
factorial(2)	2	2	2×1
factorial(1)	1	1	1×1
factorial(0)	0	1	base case

Mathematical definition:

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \times (n-1)! & \text{if } n > 0 \end{cases}$$

Recursion mirrors this definition directly.

AP Exam Tip: The base case is *not* the final answer for the original call. It is the anchor that lets all the callers above it finish their multiplication. Students who stop at the base case and report 1 are making the most common recursion error on the exam.

Worked Example 2 — Sum of Array Elements

```java
// Returns sum of arr[0..n-1]
public static int arraySum(int[] arr,
                           int n) {
    if (n == 0) {
        return 0;          // base case
    }
    return arr[n-1] +
           arraySum(arr, n-1); // recursive
}

// Usage:
int[] data = {3, 7, 2, 5};
int total = arraySum(data, data.length);
// total == 17
```

Trace for `arraySum({3,7,2,5}, 4)`**:**

Call	n	Returns
`arraySum(..., 4)`	4	17
`arraySum(..., 3)`	3	12
`arraySum(..., 2)`	2	10
`arraySum(..., 1)`	1	3
`arraySum(..., 0)`	0	0 (base)

$3+7+2+5 = \mathbf{17}$

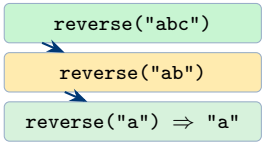
The parameter `n` decreases by 1 each call — guaranteed progress toward `n==0`.

Worked Example 3 — Recursive String Reversal

```java
public static String reverse(String
    s) {
    if (s.length() <= 1) {
        return s;           // base
            case
    }
    // last char + reverse of front
    return s.charAt(s.length() - 1)
        + reverse(s.substring(0, s.
            length()-1));
}
// reverse("hello") == "olleh"
```

Pattern: peel the last character off, recurse on the rest, prepend the peeled character to the result. Substring shrinks each call.

Trace for `reverse("abc")`**:**



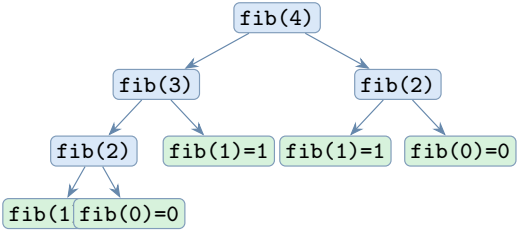
Unwind:
`reverse("ab")` = "b"+"a" = **"ba"**
`reverse("abc")` = "c"+"ba" = **"cba"**

Worked Example 4 — Fibonacci (Two Recursive Calls)

```java
public static int fib(int n) {
    if (n <= 1) {
        return n; // base cases: 0 and 1
    }
    return fib(n - 1) + fib(n - 2);
}
// fib(0)=0  fib(1)=1  fib(2)=1
// fib(3)=2  fib(4)=3  fib(5)=5
```

Two base cases: `n==0` and `n==1` are both caught by `n<=1`. **Two recursive calls** ⇒ exponential call tree, not a linear chain.

Call tree for `fib(4)`**:**



`fib(4) = 3`

Recursion vs. Iteration — Same Problem, Two Approaches

Iterative Factorial

```java
public static int factIter(int n) {
    int result = 1;
    for (int i = 1; i <= n; i++) {
        result *= i;
    }
    return result;
}
```

Recursive Factorial

```java
public static int factRec(int n) {
    if (n == 0) return 1;
    return n * factRec(n - 1);
}
```

Property	Iterative	Recursive
Code length	Usually longer	Often shorter / elegant
Stack frames	Only 1 (no extra overhead)	1 per call ($O(n)$ frames)
Risk of crash	None (loop terminates)	`StackOverflowError` if no base case
Readability	Clear for simple loops	Mirrors mathematical definition
AP exam focus	Common	**Primary focus of Unit 10**

Infinite Recursion and StackOverflowError

Broken — no base case

```
public static int broken(int n) {
    return n * broken(n - 1); // NEVER stops
}
```

Broken — base case unreachable

```
public static int alsoBreoken(int n
    ) {
    if (n == 0) return 1;
    return n * alsoBreoken(n + 1);
        // n grows!
}
```

Checklist before submitting recursive code:

- ✓ Does a base case exist?
- ✓ Is the base case reachable for all valid inputs?
- ✓ Does the recursive call use a *smaller/simpler* argument?
- ✓ Does each recursive call make progress *toward* the base case?

`StackOverflowError` at runtime = infinite recursion. Check these four items first.

AP-Style MCQ 1 — Trace the Output

```java
public static int mystery(int n) {
    if (n <= 0) {
        return 0;
    }
    return n + mystery(n - 2);
}
```

What is returned by `mystery(6)`?

- (A) 6
- (B) 9
- (C) 12
- (D) 18
- (E) 0

AP-Style MCQ 1 — Trace the Output

```java
public static int mystery(int n) {
    if (n <= 0) {
        return 0;
    }
    return n + mystery(n - 2);
}
```

What is returned by `mystery(6)`?

- (A) 6
- (B) 9
- (C) 12
- (D) 18
- (E) 0

Trace

`mystery(6)` $= 6 +$ `mystery(4)`
`mystery(4)` $= 4 +$ `mystery(2)`
`mystery(2)` $= 2 +$ `mystery(0)`
`mystery(0)` $= 0$ (base case)

Unwind:
$2 + 0 = 2$
$4 + 2 = 6$
$6 + 6 = \mathbf{12}$

Answer: (C) 12

Each call adds the current `n` and recurses with `n-2`. Steps hit even numbers: 6, 4, 2, 0.

AP-Style MCQ 2 — Identify the Output

```java
public static void printIt(int n) {
    if (n > 0) {
        printIt(n - 1);
        System.out.print(n + " ");
    }
}
// Call:  printIt(4);
```

What is printed?

- (A) 4 3 2 1
- (B) 1 2 3 4
- (C) 0 1 2 3 4
- (D) 4 3 2 1 0
- (E) Nothing is printed

AP-Style MCQ 2 — Identify the Output

```java
public static void printIt(int n) {
    if (n > 0) {
        printIt(n - 1);
        System.out.print(n + " ");
    }
}
// Call:  printIt(4);
```

What is printed?

- (A) 4 3 2 1
- (B) 1 2 3 4
- (C) 0 1 2 3 4
- (D) 4 3 2 1 0
- (E) Nothing is printed

Key Observation

The `print` happens **after** the recursive call.

So the deepest call (smallest `n`) prints first.

`printIt(4)` calls `printIt(3)` first,
which calls `printIt(2)` first,
which calls `printIt(1)` first,
which calls `printIt(0)` ⇒ base, no print.
Then unwind: prints 1, 2, 3, 4.

Answer: (B) 1 2 3 4

If `print` came *before* the recursive call, the answer would be (A). Position matters.

AP-Style MCQ 3 — What Is the Base Case?

```java
public static int sumDigits(int n) {
    if (n < 10) {
        return n;
    }
    return (n % 10) + sumDigits(n /
        10);
}
```

What does `sumDigits(284)` return?

- (A) 2
- (B) 14
- (C) 24
- (D) 284
- (E) 482

AP-Style MCQ 3 — What Is the Base Case?

```java
public static int sumDigits(int n) {
    if (n < 10) {
        return n;
    }
    return (n % 10) + sumDigits(n /
        10);
}
```

What does `sumDigits(284)` return?

- (A) 2
- (B) 14
- (C) 24
- (D) 284
- (E) 482

Trace

`sumDigits(284)`
$= (284\%10) +$ `sumDigits(28)`
$= 4 +$ `sumDigits(28)`

`sumDigits(28)`
$= (28\%10) +$ `sumDigits(2)`
$= 8 +$ `sumDigits(2)`

`sumDigits(2)`: $2 < 10 \Rightarrow$ return **2**

Unwind:
$8 + 2 = 10$
$4 + 10 = \mathbf{14}$

Answer: (B) 14

This computes the digit sum: $2 + 8 + 4 = 14$.

AP-Style MCQ 4 — Spot the Bug

```java
// Intended: return n^k (n to the kth power)
public static int power(int n, int k) {
    if (k == 0) {
        return 0;            // LINE A
    }
    return n * power(n, k - 1);
}
```

Which line has the bug? What should it be?

- (A) Line A: change `return 0` to `return 1`
- (B) Line A: change `k == 0` to `k == 1`
- (C) Recursive call: change `k - 1` to `k + 1`
- (D) Recursive call: change `n *` to `n +`
- (E) There is no bug

AP-Style MCQ 4 — Spot the Bug

```
// Intended: return n^k (n to the kth power)
public static int power(int n, int k) {
    if (k == 0) {
        return 0;                    // LINE A
    }
    return n * power(n, k - 1);
}
```

Which line has the bug? What should it be?

- (A) Line A: change `return 0` to `return 1`
- (B) Line A: change `k == 0` to `k == 1`
- (C) Recursive call: change `k - 1` to `k + 1`
- (D) Recursive call: change `n *` to `n +`
- (E) There is no bug

Analysis

$n^0 = 1$ for any n. Base case should return 1, not 0.

With `return 0`: all calls multiply by 0.

Test: `power(3,2)`
$= 3\times$`power(3,1)`$= 3\times 3\times$`power(3,0)`
$= 3\times 3\times 0 = 0$ **(wrong)**

With `return 1`: $= 3\times 3\times 1 = 9$ ✓

Answer: (A)

The base case of exponentiation is $n^0 = 1$, not 0.

FRQ-Style Practice — Write the Recursive Method

Problem

Write a static recursive method `countChar(String s, char c)` that returns the number of times `c` appears in `s`.

Examples: `countChar("banana",'a')` $\rightarrow$ 3; `countChar("hello",'z')` $\rightarrow$ 0

FRQ-Style Practice — Write the Recursive Method

Problem

Write a static recursive method `countChar(String s, char c)` that returns the number of times `c` appears in `s`.

Examples: `countChar("banana",'a')` → 3; `countChar("hello",'z')` → 0

```
public static int countChar(String s
    , char c) {
    if (s.length() == 0) {
        return 0;           // base
            case
    }
    int match = (s.charAt(0) == c) ?
        1 : 0;
    return match + countChar(s.
        substring(1), c);
}
```

AP FRQ rubric points:

- Base case: `length()==0`, return 0
- Correctly checks first char
- Recursive call on `s.substring(1)`
- Returns sum of match + recursive result

`s.substring(1)` removes the first character — string shrinks each call, reaching length 0.

Common AP Exam Traps — and How to Avoid Them

Trap	**How to avoid it**
"The base case return value *is* the answer"	Trace all the way back up. Callers above the base case still have multiplications / additions to do.
Forgetting `print` order depends on placement	`print` *before* recursive call = top-down order. `print` *after* = bottom-up (reversed).
Two base cases? One might be wrong	Check *both* conditions when tracing. `n <= 1` covers both `n==0` and `n==1`.
"It never terminates" panic	Check: is the argument strictly smaller (or simpler) each call?
Base case returns wrong literal	$n^0 = 1$, not 0. $0! = 1$, not 0. Empty string sum = 0.

Lesson Summary

Core Concepts

- A recursive method **calls itself**
- **Base case**: stops recursion, returns without calling self
- **Recursive case**: calls self with a *simpler* argument
- Each call gets its own **stack frame** with its own variables
- **Winding down**: calls accumulate
- **Unwinding**: return values propagate back up
- No base case (or unreachable) ⇒ `StackOverflowError`

AP Exam Tips

- **Always trace on paper.** Do not try to picture it mentally.
- Write call, check base case, indent recursive calls.
- Work **back up** from the base case to find return values.
- Note where `print`/`return` is relative to the recursive call.
- 4–6 recursion MCQs per exam — every one counts.

Next lesson: Recursive searching (binary search) and sorting (Mergesort) — Unit 10 continues.

Exit Card

Answer all four — on paper before leaving

(a) What are the two required components of any recursive method?

(b) What is returned by this call? Trace your answer.

```
public static int f(int n) {
    if (n == 1) return 10;
    return f(n - 1) + 5;
}
// f(4) = ?
```

(c) What Java error occurs when recursion never reaches its base case?

(d) What is wrong with this base case?

```
public static int power(int n,
    int k) {
    if (k == 0) return 0; //
        ???
    return n * power(n, k - 1)
        ;
}
```

Exit Card

Answer all four — on paper before leaving

(a) What are the two required components of any recursive method?

(b) What is returned by this call? Trace your answer.

```
public static int f(int n) {
    if (n == 1) return 10;
    return f(n - 1) + 5;
}
// f(4) = ?
```

(c) What Java error occurs when recursion never reaches its base case?

(d) What is wrong with this base case?

```
public static int power(int n,
    int k) {
    if (k == 0) return 0; //
        ???
    return n * power(n, k - 1)
        ;
}
```

Answers: (a) Base case and recursive case. (b) `f(4)=25: f(4)=f(3)+5=f(2)+10=f(1)+15=25` (c) `StackOverflowError` (d) Should return `1` (since $n^0 = 1$, not 0)